MedBridge Patient Agent

Team Task Breakdown · Healtour Solutions / Kiron Group

Project	MedBridge — medical-tourism AI agent (intake → Journey Plan)
Hackathon	Google for Startups AI Agents Challenge 2026 · Track 1: Build
Deadline	June 5, 2026 · 5:00 PM PT — submit early
Company	Healtour Solutions / Kiron Group

0. Locked Architecture Decisions

■■ Stack: **Gemini + Google ADK** (not Claude + LangGraph). Uses $500 Google Cloud credits and satisfies the hackathon Mandatory Technologies rule.

Concern	Decision
LLM (Intelligence)	Gemini (gemini-2.0-flash) via Vertex AI
Orchestration	Google ADK (Python)
Tools	Python function tools on the ADK agent (JSON knowledge base lookups)
Agent Flow	hospital_match → parallel(cost_estimator, document_checklist) → synthesise
Backend	ADK + FastAPI → Cloud Run (1 service)
Frontend	React + Tailwind → Cloud Run (1 service), live streaming via SSE
Data	3 JSON flat files (hospital registry, cost matrix, country-doc rules)
Secrets	No LLM API key — Gemini-on-Vertex uses Cloud Run service account

Mandatory-tech checklist (all 3 must be true to qualify):

- ■ Intelligence = Gemini
- ■ Orchestration = ADK
- ■ Infrastructure = deployed on Cloud Run

1. Repo Structure

Shyam owns; everyone branches off it. Git workflow: *main* is protected. Each person works on *feat/<name>-<thing>* and opens a PR.

```
medbridge/
■■■ backend/                  (Team Alpha)
■   ■■■ medbridge_agent/
■   ■   ■■■ agent.py          ADK agent + orchestration   (SHYAM)
■   ■   ■■■ contract.py       shared tool schemas/types   (SHYAM) ← build first
■   ■   ■■■ tools/
■   ■       ■■■ hospital_match.py     (SHYAM + RAM)
■   ■       ■■■ cost_estimator.py     (GOPI)
■   ■       ■■■ document_checklist.py (PALLAVI)
■   ■■■ data/                 Knowledge base              (RAM)
■   ■   ■■■ hospital_registry.json
■   ■   ■■■ cost_matrix.json
■   ■   ■■■ country_doc_rules.json
■   ■■■ main.py               FastAPI /run + SSE          (SHYAM)
■   ■■■ requirements.txt
■   ■■■ Dockerfile            (RAM + SHYAM)
■■■ frontend/                 (Team Bravo)
```

```
■   ■■■ src/
■   ■   ■■■ IntakeForm.jsx    (HARI)
■   ■   ■■■ JourneyPlanCard.jsx (VENKAT)
■   ■   ■■■ ErrorBoundary.jsx (OMKAR)
■   ■   ■■■ api.js            SSE client (HARI)
■   ■■■ Dockerfile            (OMKAR)
■■■ TEAM_TASKS.md
■■■ README.md
```

2. The API Contract

Everyone codes to this. **origin_country is always a 2-letter ISO code** chosen from a dropdown — never free text. This is the #1 silent bug.

Request — frontend → backend:

```
POST /run
{ "condition": "knee replacement", "origin_country": "AE", "budget_usd": 8000, "timeline_weeks": 6 }
```

Response — SSE stream (text/event-stream):

```
data: {"type":"token","text":"Based on your profile, ..."}
data: {"type":"hospital","name":"Apollo Hospitals","city":"Hyderabad","tier":"A"}
data: {"type":"cost","total_usd":6800,"breakdown":[...]}
data: {"type":"docs","items":["Passport","Medical records","Visa letter"]}
data: {"type":"done"}
```

3. Per-Person Assignments

■ TEAM ALPHA — Backend / Agent

SHYAM — Agent Lead & Orchestration

What you own: The agent brain, repo scaffold, shared contract, API layer, backend deploy, and integration.

What you build:

- Repo scaffold + contract.py (tool schemas) — Day 1 AM, first thing.
- ADK agent + orchestration flow.
- hospital_match tool (with Ram).
- FastAPI /run endpoint with SSE streaming.
- Backend Cloud Run deploy.

How to build it:

- pip install google-adk fastapi uvicorn.
- contract.py: define tool inputs/outputs as Pydantic models + clear docstrings. Commit immediately.
- agent.py: build a deterministic ADK workflow — hospital_match node → ParallelAgent(cost_estimator + document_checklist) → LlmAgent(Gemini) synthesise. Wrap as SequentialAgent.
- Test locally with adk web using the Rahul case before any frontend exists.
- main.py: FastAPI POST /run, streams SSE, sends PING keepalive every ~20s, enables CORS.
- Deploy: adk deploy cloud_run or Docker + gcloud run deploy medbridge-backend --timeout 300 --allow-unauthenticated.

RAM — Knowledge Base & Backend Deploy — Day 1 dependency gate

What you own: All 3 JSON data files — the 'moat.' Everything in Layer 3 waits on this.

What you build:

- hospital_registry.json — 22 hospital records.
- cost_matrix.json — costs per hospital × procedure.

- country_doc_rules.json — visa/doc rules per country.
- Co-own backend Cloud Run deploy with Shyam.

How to build it:

- Use Shyam's contract.py schema. Finish + get Shyam's review by end of Day 1.
- hospital_registry.json: ≥5 specialty tags each, mou_active:true to appear, unique id.
- cost_matrix.json: keyed hospital_id → procedure-slug → { total_usd_min/max, total_inr_min/max, line_items }.
- country_doc_rules.json: keyed by ISO code → { visa_required, visa_type, mandatory_docs[], recommended_docs[] }.
- Write a tiny validate.py that loads each file and checks required fields.
- Data loads once at startup (module-level variable), not per request.

GOPI — Cost Estimator Tool

What you own: The cost_estimator tool.

What you build:

- cost_estimator(hospital_id, procedure, origin_country) → CostBreakdown

How to build it:

- Import schema from contract.py.
- Read cost_matrix.json, look up hospital_id × procedure, sum line_items, return total_usd + total_inr.
- Hardcode FX rate (June 2026) — no live FX API.
- Apply origin-country surcharge if relevant.
- Never return NaN/None — if unknown, return 0 with confidence:'approximate' flag.
- Write a clear docstring — ADK feeds it to Gemini as the tool description.

PALLAVI — Document Checklist Tool — build first, lowest risk

What you own: The document_checklist tool.

What you build:

- document_checklist(origin_country, hospital_id) → DocChecklist

How to build it:

- Import schema from contract.py.
- Read country_doc_rules.json, look up ISO country code, return mandatory + recommended docs + visa info.
- Unknown country → generic fallback message — never an error, never empty.
- Pure static logic, no external calls. Build this first on Day 1 to prove the ADK → tool chain.
- Write a clear docstring for Gemini.

■ TEAM BRAVO — Frontend / Demo

VENKAT — Frontend Lead · Journey Plan Renderer + Demo Video

What you own: The result UI and the demo experience.

What you build:

- JourneyPlanCard.jsx — renders 3 sections: hospital, cost breakdown, document checklist.
- Loading skeleton (appears instantly on submit).
- Co-own the 90-sec demo video.

How to build it:

- React + Tailwind. Render from agent JSON by key name, never by array index.
- Wrap any JSON.parse in try/catch — on failure, show raw narrative text, never a blank screen.
- Skeleton card renders the moment the user submits; SSE tokens stream in and fill it live.

• Video script: "Rahul, 54, Dubai, knee replacement, quoted $22k" → fill form → skeleton → plan streams → "$6,800, 3 docs, saved $15k".

HARI — Intake Form + SSE Client

What you own: Data in (the form) and stream in (the SSE client).

What you build:

• IntakeForm.jsx — controlled form, 4 fields.

• api.js — the SSE client that talks to the backend.

How to build it:

• Controlled React inputs: condition (text), origin_country (ISO dropdown, not free text), budget_usd (number), timeline_weeks (number).

• On submit, POST /run to the backend Cloud Run URL, then open an EventSource to consume the SSE stream.

• Show the loading skeleton immediately on submit.

CHANDRIKA — Validation + Devpost Submission Owner

What you own: Input correctness AND the final submission (critical — submission is a deliverable).

What you build:

• Client-side form validation rules.

• Polished Journey Plan card sections (with Venkat).

• The Devpost submission page + business-case writeup (30% of the score).

How to build it:

• Validation: all fields required; budget_usd > 0; condition min 3 chars; timeline_weeks ≥ 2. Show inline errors before network call.

• Business case: write the problem + ROI story — "medical tourism saves 60–80%; MedBridge cuts research from days to seconds; Rahul saved $15,000."

• Assemble the 4 deliverables on Devpost: code link, architecture PNG, demo video, live URL. Pick Theme = Build. Submit a day early.

OMKAR — Error Handling + Frontend Deploy + Config

What you own: Resilience and getting the frontend live.

What you build:

• ErrorBoundary.jsx — catches agent timeout / network failure, shows a retry button.

• Frontend Cloud Run deploy.

• Config / any secrets.

How to build it:

• Error boundary: on any failure, show friendly message + retry — never a blank screen (judges will see it).

• Dockerfile: build React app, serve via nginx on port 8080; proxy /api to the backend URL.

• gcloud run deploy medbridge-frontend → confirm the URL opens in an incognito window.

• No LLM API key to manage. Ensure the backend Cloud Run service account has the Vertex AI User role.

4. Day-by-Day Timeline

Day	Team Alpha (Backend)	Team Bravo (Frontend)
Day 1	Shyam: scaffold + contract.py + agent skeleton in adk web. Ram: 3 JSON files + Shyam review. Pallavi: doc tool (first). Gopi: cost tool.	Hari: intake form. Venkat: Journey card + skeleton. Chandrika: validation. Omkar: error boundary.

Day 2	Finish agent flow + parallel tools + FastAPI/SSE → deploy backend to Cloud Run.	Wire frontend to backend URL → deploy frontend → CORS check. 8 PM integration checkpoint (Shyam + Venkat).
Day 3	Polish, edge cases, arch diagram PNG (Shyam), make repo judge-accessible.	Record video (Venkat + Omkar). Chandrika fills Devpost. Submit early.

5. Day-2 Integration Checkpoint

All 7 checks must pass:

1. Backend /run live on Cloud Run, returns 200 for a test intake. (Ram)
2. All 3 tools return valid JSON matching contract.py. (Shyam)
3. Frontend submits to the Cloud Run URL (not localhost). (Venkat)
4. SSE stream renders live — no blank pause. (Hari)
5. Journey card shows hospital + cost + docs. (Chandrika)
6. Error state renders cleanly. (Omkar)
7. Demo URL opens in incognito. (Venkat)

■ If any check fails by Day-2 evening, escalate to Prasanthi immediately. A known blocker with 12 hours left is recoverable; an unknown one on Day 3 morning is not.

6. The 4 Submission Deliverables

#	Deliverable	Owner
1	Source code (GitHub, public or judge access)	Shyam
2	Architecture diagram (PNG)	Shyam
3	90-sec demo video ("Rahul")	Venkat + Omkar
4	Live Cloud Run demo link (test incognito!)	Omkar / Ram

Single source of truth for the MedBridge build. Stack: Gemini + ADK + Cloud Run. Questions → Shyam (backend) / Venkat (frontend) / Prasanthi (escalation).